

Лабораторная работа №9: Глобальное UI-состояние и композиция данных (Recoil)

Цель работы: Дополнить проект, основанный на TanStack Query (ЛР 8), слоем локального клиентского состояния для управления интерфейсом, используя Recoil.

Задание

1. Архитектура состояния

Настройте RecoilRoot в App.tsx.

- **Важно:** Вы должны четко разграничить: всё, что касается списка товаров — остается в useQuery. Всё, что касается **интерфейса пользователя** — выносится в атомы Recoil.

2. Реализация UI-настроек

Реализовать возможность кастомизации интерфейса:

- Создать атом uiSettingsState, содержащий:
 - viewMode (значения: 'grid' (Режим сетки) или 'list' (Режим списка)).
 - theme (значения: 'light' или 'dark').
- В компоненте CatalogPage добавить переключатели, которые меняют эти настройки.
- Использовать селектор, чтобы адаптировать классы отображения списка товаров в зависимости от viewMode.

3. Реализация управления состоянием «Корзина» и «Избранное»

Цель: Реализовать клиентскую бизнес-логику управления товарами без обращений к API, обеспечив мгновенный отклик интерфейса (Optimistic UI).

1. Моделирование состояния (Atoms)

Разделить данные на два типа:

- **Избранное (favoritesState):** Атом, хранящий массив ID товаров (number[]). Это простая логика «добавить/убрать».
- **Корзина (cartState):** Атом, хранящий объект или массив объектов вида { id: number, quantity: number }. Это более сложная логика, требующая управления количеством (инкремент/декремент).

2. Вычисляемое состояние (Selectors)

- **Счетчик (cartCountState):** Селектор, возвращающий сумму всех quantity в корзине.
- **Детализация данных (cartProductsSelector):** Продвинутый селектор, который объединяет данные: берет ID из Recoil-атома и сопоставляет их с полными объектами товаров, полученными из TanStack Query.

3. Интерфейс (UI)

Для работы с состоянием реализовать следующие UI-компоненты:

- **Catalog Page:**
 - Добавить на карточку товара кнопку «В избранное» (иконка сердца). Состояние кнопки (активна/неактивна) должно зависеть от наличия ID товара в `favoritesState`.
 - Добавить кнопку «В корзину». При клике товар должен добавляться в `cartState`.
- **Navbar:**
 - Реализовать «Badge» (счетчик) поверх иконки корзины, который отображает значение `cartCountState`. Счетчик должен обновляться мгновенно при любом изменении корзины.
- **Cart Page:**
 - Создать отдельную страницу, где пользователь видит список добавленных товаров.
 - Реализовать возможность изменения количества (+ / -) и удаления товара (`remove`).

4. Технические требования

- **Производительность:** Используйте `useSetRecoilState` в кнопках «Добавить/Удалить», чтобы избежать лишних ререндеров родительских компонентов карточек.
- **Изоляция:** Логика добавления в корзину **не должна** вызывать сетевых запросов. Все операции происходят исключительно в памяти браузера.
- Реализуйте простую синхронизацию с `localStorage` через `effects` в атомах Recoil, чтобы корзина не пропадала при перезагрузке страницы.
- Для каждой компоненты добавьте кнопку «Очистить все», на панели навигации - «Сбросить настройки» (использовать `useRecoilCallback`)

Контрольные вопросы:

1. Почему мы храним ID в Recoil, а не полные объекты товаров?
2. Как компоненты узнают, что товар уже в корзине?
3. Как передать состояние корзины в оформление заказа?
4. Почему мы храним `products` (список товаров) в TanStack Query, а `viewMode` (вид отображения) — в Recoil?
5. Что произойдет, если мы попробуем хранить товары в Recoil (потеряем ли мы возможности кэширования и инвалидации)?
6. Как Recoil гарантирует, что Navbar будет перерисован **только** при изменении `cartCountState`?
7. Как Recoil гарантирует это отсутствие ререндера?
8. В чем разница между использованием `useRecoilState` и `useSetRecoilState` в компоненте переключателя темы?
9. Какой из них является более производительным и почему?
10. Можно ли создать селектор, который зависит одновременно от данных из `useQuery` и данных из атома Recoil?